

# Fagprøve, del 1: Asjer

---

## Oppgave 1: Om bedriften og læretiden

### Asjer:

- 19 år
- Født og oppvokst i Norge, med bakgrunn fra Eritrea
- Fant ut at jeg liker utvikling ved et uhell

### Digitaliseringsetaten:

- Tidligere kjent som UKE, nå DIG
- Leverer IT-løsninger for virksomheter, bydeler og innbyggere i Oslo kommune
  - oslo.kommune.no
  - Bil i Oslo
  - bevillinger.app.oslo.kommune.no

### Hovedoppgaver:

- Planlegge, utvikle og vedlikeholde løsninger som brukes av virksomheter og bydeler i Oslo kommune:
  - Spesialpedagogisk App (BGA / Bydel Grünerløkka)
  - Bestandsoversikt (VEL / Velferdsetaten)
  - OppgaveKlokka (VEL / Velferdsetaten)

### Utviklingsrelaterte praksiser og rutiner:

Det meste av det vi gjør samsvarer med det man finner i andre utviklingsmiljøer. Utvikling skjer i:

- Test-, PreProd- og Prod-miljøer

Man utvikler lokalt, tester endringene i testmiljøet, deretter i PreProd, og til slutt i prod etter en code review.

Vi har også faglige innlegg som skal bidra til å hindre "bussfaktoren". Det vil si at én på teamet deler noe faglig nyttig med resten. Det kan være noe relatert til kodebasene vi jobber i, ulike namespaces, eller utviklingsverktøy de har oppdaget som gjør arbeidsdagen enklere.

Et eksempel er fzf-finder. En på teamet holdt et faglig innlegg om dette terminalverktøyet, som holder historikk over alle kommandoer du har kjørt.

## Lærlingtiden min:

---

### Spesialpedagogisk app (BGA)

Bydel Grünerløkka (BGA) sendte inn en bestilling til oss i DIG hvor de ønsket hjelp med å utvikle en løsning for å føre timer på barn som trenger spesialpedagogisk hjelp.

BGA førte all denne dataen i Excel-ark og Word-dokumenter, som ble delt rundt med teamet. Alle jobbet i et felles dokument, noe som var både tungvint og generelt bad practice.

Kunden kom med spesifikke krav til hva løsningen skulle inneholde:

1. Man skal kunne registrere nye barn i løsningen med en Geric-ID
2. Man skal kunne føre timer på barnet
3. BGA jobbet i forskjellige team, så appen måtte støtte opprettelse av team
4. Løsningen skal ha tilgangsstyring:
  - Admin: Full tilgang og kan gjøre alt
  - Teamleder: Ansvar for et team og alle støtteressursene som tilhører det teamet
  - Ansatt: En vanlig støtteressurs med ansvar for ett eller flere barn

I første omgang ønsket kunden mulighet til å føre initialer på et barn. Dette var greit, men det førte til at BGA måtte gjennomføre en ROS-analyse (Risiko- og Sårbarhetsanalyse) av løsningen. Det stilte også krav til oss som utviklere, om å sikre at kun de med tilgang til et barn kan se data og føre timer på det.

Geric-ID kan teknisk sett falle under GDPR (General Data Protection Regulation), men alt løsningen vår eksponerte var Geric-ID-en. Ønsket man mer informasjon, måtte man ha tilgang til Geric-systemet og der igjen tilgang til den spesifikke Geric-ID-en.

### Bestandsoversikt (VEL)

---

Velferdsetaten (VEL) ønsket en løsning hvor de kunne ha oversikt over arkivet sitt. På lik linje med BGA brukte VEL Word-dokumenter.

De kom til oss på et Power-verksted hvor de pitchet ideen, og sendte inn en Kompass-sak (sakssystemet vårt).

I løsningen skulle det være mulig å gjøre endringer på et element, og deretter skulle en leder ha muligheten til å godkjenne eller avvise endringen. Problemet var at når en ansatt gjorde en endring og sendte den til godkjenning, fikk ikke admin noe varsel om at en sak ventet på godkjenning.

Det er her Power Automate kommer inn. Når en ny sak kommer inn og trenger godkjenning, sendes det et varsel til en admin via e-post som informerer dem om at en sak venter på godkjenning.

Men hvordan fungerer flyten?

Når en ansatt sender inn en sak eller gjør en endring på en eksisterende sak og ber om godkjenning, gjøres det et kall mot databasen som sjekker om et element er blitt endret. I Power Automate kjøres det en condition som sjekker om et element er blitt endret i appen. Hvis ja, sendes det en e-post til en admin. Hvis nei, skjer det ingenting.

## Designmanual for Power App løsninger (Oslo Kommune)

---

Ved utvikling av fullkodeløsninger i Oslo Kommune bruker utviklere Punkt-designsystemet. Dette gjør det enkelt å utvikle raskt og effektivt, og sparer mye tid.

Power Apps manglet noe tilsvarende. Når en utvikler skal lage en løsning i Power Apps, må man designe komponentene selv i henhold til Oslo Kommunes retningslinjer. Vi så derfor et behov for å utvikle et komponentbibliotek for Power Apps-løsninger.

Komponentene følger Oslo Kommunes retningslinjer, så langt det lar seg gjøre, ettersom Power Apps kan ha enkelte begrensninger når det kommer til design. Biblioteket er plug and play: man velger et komponent man trenger, trykker på kopier-knappen som tilhører komponenten, og limer det inn i løsningen.

Vi sikrer også at utviklere faktisk bruker det. Alle løsninger som utvikles i Oslo Kommune skal se ut som "Oslo Kommune", og under code review oppdager man raskt om noen ikke har brukt komponenter fra biblioteket.

Merk at komponent biblioteket er i en beta/testing fase, og enda ikke rullet ut til alle virksomheter i Oslo Kommune.

## Lavkode vs. fullkode

---

Lavkode og fullkode er to mye brukte verktøy, både i Oslo Kommune og i andre offentlige og private sektorer. Det er imidlertid noen konkrete og store forskjeller mellom dem.

### **Lavkode:**

Lavkode er noe man kan sette sammen raskt. Man utvikler på ferdig infrastruktur fra Microsoft, hvor autentisering, tilgangsstyring og sikkerhet allerede er på plass. Det er mye man får "out of the box", spesielt om man utvikler i Power Platform.

Det finnes likevel en del begrensninger. Alt skal som regel skje innenfor Power Platform og Microsofts univers. Man har ikke fritt valg av database, API-kallene man kan gjøre er begrenset, og man er som regel låst til ett programmeringsspråk: Power FX.

### **Fullkode:**

Fullkode er det mest vanlige ute i markedet. Det er fleksibelt og har ikke like strenge grenser for hva som er mulig. Man kan velge akkurat den stacken som passer løsningen sin, inkludert rammeverk, infrastruktur, skyplattform og mye mer. Men alt dette må settes opp selv. Man

kan få en skyplattform levert, men integrasjon og konfigurasjon mot applikasjonen må man gjøre helt selv.

Med lavkode får man som regel alt servert på et sølvfat, men med begrensninger. Med fullkode har man full valgfrihet, men må sette opp alt selv.